

If statement with initializer

Abstract

We propose a new version of the `if` statement for C++: `if (init; condition)`. This statement simplifies common code patterns and helps users keep scopes tight.

Contents

1. [Revision history](#)
2. [Before/After](#)
3. [Proposal](#)
4. [Motivation](#)
5. [Alternatives](#)
6. [Discussion](#)
7. [Impact on the Standard](#)
8. [Proposed wording](#)
9. [Future directions](#)

Revision history

P0305r0: First draft

Before/After

Before the proposal	With the proposal
<pre> { auto p = m.try_emplace(key, value); if (!p.second) { FATAL("Element already registered"); } else { process(p.second); } } </pre>	<pre> if (auto p = m.try_emplace(key, value); !p.second) { FATAL("Element already registered"); } else { process(p.second); } </pre>
<pre> status_code foo() { { status_code c = bar(); if (c != SUCCESS) { return c; } } // ... } </pre>	<pre> status_code foo() { if (status_code c = bar(); c != SUCCESS) { return c; } // ... } </pre>
<pre> void safe_init() { </pre>	<pre> void safe_init() { </pre>

```

{
    std::lock_guard<std::mutex>
lk(mx_);
    if (v.empty())
        v.push_back(kInitialValue);
}
}
// ...
}

```

```

if (std::lock_guard<std::mutex>
lk(mx_); v.empty()) {
    v.push_back(kInitialValue);
}
// ...
}

```

(Consider having to move this code around.)

Proposal

There are three statements in C++, `if`, `for` and `while`, which are all variations on a theme. We propose to make the picture more complete by adding a new form of `if` statement.

Statement	Equivalent to*	Iterations
<code>while(cond) E;</code>	<code>{ while(cond) { E; } }</code>	Repeatedly while <code>cond</code> holds
<code>for (init; cond; inc) E;</code>	<code>{ init; while(cond) { E; inc; } }</code>	Repeatedly while <code>cond</code> holds
<code>if (cond) E;</code>	<code>{ while(cond) { E; break; } }</code>	Once while <code>cond</code> holds
<code>if (cond) E; else F;</code>	(more complex) [†]	Once
<code>if (init; cond) E;</code> new!	<code>{ init; while(cond) { E; break; } }</code>	Once while <code>cond</code> holds
<code>if (init; cond) E; else F;</code> new!	(more complex) [†]	Once

Notes:

*) The “equivalence” ignores the fact that `break` and `continue` have different semantics in loops.

†) The fact that there is no immediate expression of `else` blocks in terms of `while` is due to the absence of a fundamental `while ... else` construction from the language, which would in some sense be a “universal control structure”.

Motivation

The new form of the `if` statement has many uses. Currently, the initializer is either declared before the statement and leaked into the ambient scope, or an explicit scope is used. With the new form, such code can be written more compactly, and the improved scope control makes some erstwhile error-prone constructions a bit more robust:

```

std::map<int, std::string> m;
std::mutex mx;
extern bool shared_flag; // guarded by mx

int demo()
{
    if (auto it = m.find(10); it != m.end()) { return it->size(); }
}

```

```

    if (char buf[10]; std::fgets(buf, 10, stdin)) { m[0] += buf; }

    if (std::lock_guard<std::mutex> lock(mx); shared_flag) { unsafe_ping(); shared_flag =
false; }

    if (int s; int count = ReadBytesWithSignal(&s)) { publish(count); raise(s); }

    if (auto keywords = {"if", "for", "while"};
        std::any_of(keywords.begin(), keywords.end(), [&s](const char* kw) { return s == kw;
})) {
        ERROR("Token must not be a keyword");
    }
}

```

A certain “monadic” style of bubbling up non-success status values also becomes more compact:

```

status_code bar();

status_code foo()
{
    int n = get_value();
    if (status_code c = bar(n); c != status_code::SUCCESS) { return c; }
    if (status_code c = do_more_stuff(); c != status_code::SUCCESS) { return c; }
    return status_code::SUCCESS;
}

```

Alternatives

No extension

There is always the alternative of using an equivalent construction, namely `{ init; if (cond) E; }`. However, this construction is more verbose, and users often use a “lazy approximation” that omits the extra scope. such as:

```

auto it = m.find(10);
if (it != m.end()) { return it->size(); }

// "it" is leaked into the ambient scope.

```

This is often just as good, but in certain cases where the lifetime of the object created in the initializer is important, such as when locking a mutex, forgetting the extra scope may easily have hard-to-diagnose adverse effects. Moreover, the explicit additional scope is brittle and may get lost during refactoring.

A common naming convention is that the length of a name of should correspond to the size of its scope; offering a convenient way to declare names in tight scopes makes it easier to follow such a principle without either introducing unwanted nesting levels or using artificially long names.

Library solution

It is possible to write a library gadget that could contain both initialized values and the result of a boolean expression, but all such attempts have turned up something very unsightly.

Other language extensions

An alternative language extension could be of the form `with (init) if (cond) E;`. Constructions like this exist in other languages. While certainly conceivable, such an extension is more expensive (new keyword, more to teach) and misses out on the opportunity to make an existing facility more consistent.

Discussion

It is often said that C++ is already complex enough, and any additional complexity needs to be carefully justified. We believe that the proposed extension is natural and unsurprising, and thus adds minimal complexity, and perhaps even removes some of the existing differences among the various control flow statements. There is nothing about the local initialization that is specific to loop statements, so having it only on the loop and not on the selection statement seems arbitrary. Had the initializer form of the `if` statement been in the language from the start, it would not have seemed out of place. (At best one might have wondered why `for` is not also spelled `while`, or vice versa.)

For the second point, we would like to consider the advantages of the new form of the `if` statement. Names, lifetimes and scopes are fundamental concepts of C++, and putting the right names into the right scopes is instrumental to understandability and maintainability. The proposed extension is mere syntactic sugar, but it is a convenient tool, readily understood by the reader, that allows the user to keep the scope of auxiliary variables minimal. Current code either requires additional braces to keep scopes minimal, which are visually noisy (taking up valuable indentation levels!) and burdensome to refactor (think of keeping all the lines together), or simply omits the braces, leaking local variables into larger scopes.

Real, existing code bases contain macros that wrap up common idioms (like map lookup and error status propagation), because users find macros the smaller of the two evils compared to leaking lots of local variables or using excessive braces. The proposal removes a common use case for macros.

Impact on the Standard

This is a core language extension. The newly proposed syntax is ill-formed in the current working draft.

Proposed wording

In section 6.4, change the grammar in paragraph 1 as follows.

selection statement:

```
if ( condition ) statement
if ( condition ) statement else statement
if ( for-init-statement condition ) statement
if ( for-init-statement condition ) statement else statement
switch ( condition ) statement
```

Insert a new paragraph at the beginning of subsection 6.4.1.

The if statement

```
if ( for-init-statement condition ) statement
```

is equivalent to

```
{
    for-init-statement
    if ( condition ) statement
}
```

and

```
if ( for-init-statement condition ) statement else statement
```

is equivalent to

```
{
```

```
for-init-statement  
if ( condition ) statement else statement  
}
```

except that names declared in the *for-init-statement* are in the same declarative region as those declared in the *condition*.

Future directions

Yet more statements

There is another statement which is related to `if`, `for` and `while`: `switch`. The proposed extension could just as easily be applied to `switch`:

```
switch (Foo x = make_foo(); x.status()) {  
    default: /* ... */  
    case Foo::FINE: /* ... */  
    case Foo::GOOD: /* ... */  
    case Foo::NEAT: /* ... */  
}
```

The only reason that we are not proposing that extension here is that use cases seem much fewer than for the `if` statement and that we would like to keep this proposal small. Note also that the Go language allows initial statements for both `if` and `switch` statements.

Sneak peak: `constexpr if`

There is currently an extension proposal on its way for augmenting the existing `if` statement with an optional `constexpr` specifier (e.g. [P0128](#), P0292). While we cannot speak to the impact of un-moved proposals, we imagine that the present extensions would probably *not* also obtain an optional `constexpr` specifier, mostly for reasons of simplicity and for lack of use cases. If the specifier were to be allowed, presumably the *for-init-statement* would need to be constrained to be define a constant expression (and not be an assignment). Details of the interaction of the two proposals may be addressed in the future.